Lab 9: User-Defined Functions and Menu-Driven Programs

Topics covered:

- User-defined functions
- Function prototypes and definitions
- Function calls, arguments, and parameters
- void and value-returning functions
- Menu-driven programs
- Loops and conditional statements

Learning Objectives

By the end of this lab, students should be able to:

1. Define and call user-defined functions.
2. Distinguish between arguments and parameters.
3. Write function prototypes and definitions.
4. Explain the difference between void and value-returning functions.
5. Use functions with if/else, switch, and loops.
6. Develop a menu-driven program using multiple functions.

Review: What Is a User-Defined Function?

- A user-defined function is a collection of statements that performs a specific task.
- General structure:

```
returnType functionName(parameters)
{
    // Statements
}
```

- Example:

```
void displayMessage()
{
    cout << "Welcome to CIS 123!" << endl;
}
```

- Calling the function:

```
int main()
{
    displayMessage();

    return 0;
}
```

- Does defining a function automatically execute it?

No. A function must be called.

Function Prototypes, Definitions, and Calls

```
#include <iostream>
using namespace std;

// Function prototype
void displayMessage();

int main()
{
    displayMessage();  // Function call

    return 0;
}

// Function definition
void displayMessage()
{
    cout << "Hello, students!" << endl;
}
```

Important concepts:

- Prototype: Informs the compiler about the function.
- Definition: Contains the statements executed by the function.
- Call: Transfers control to the function.
- After execution, control returns to the calling function.

Why do we need a prototype when the function is defined after mai

We need a function prototype because the C++ compiler must know that a function exists before it encounters a call to that function.

Arguments and Parameters

```
void displaySum(int x, int y)
{
    cout << "Sum: " << x + y << endl;
}

int main()
{
    int a = 10;
    int b = 20;

    displaySum(a, b);

    return 0;
}
```

Argument	Parameter
a	x
b	y
Supplies a value	Receives a value

Important: Argument names do not need to match parameter names, but their order and compatible types matte

`Void` vs. Value-Returning Functions

- Void function

```
void add(int a, int b)
{
    cout << a + b;
}
```

- Displays a result but does not return a value.

- Value-returning function

```
int add(int a, int b)
{
    return a + b;
}
```

- Returns a value that the caller can store or displa

- Can we assign the result of a void function to an integer variable?

No.

Menu-Driven Calculator

Write a C++ program that displays a menu and performs arithmetic operations based on the user's choice.

The program should display:

```
===== CALCULATOR MENU =====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit

Enter your choice:
```

Program requirements:

- Create a separate void function for each arithmetic operation.
- Use a do-while loop to repeat the menu.
- Use switch to process the user's choice.
- Ask for two integers when options 1–4 are selected.
- Prevent division by zero.
- Exit only when the user selects option 5.

Think Before You Code:

What operations should our calculator perform? Addition, subtraction, multiplication, and division.

Which operations should be implemented as separate functions? showMenu(), add(), subtract(), multiply(), and divide

What parameters should each arithmetic function receive? Each arithmetic function receives two integers (int a, int b

Should these functions return a value or display the result using void? Use void because each function displays its result rather than returning a value.

Which function should display the menu? showMenu() displays the available operations.

How can we repeatedly display the menu until the user chooses Exit? Use a do-while loop to repeat the menu until the user selects 5.

How should the program handle division by zero and invalid menu choices? Use if to prevent division by zero and default in a switch statement to handle invalid menu choices.

- Expected Function Prototypes

```
void showMenu();
void add(int, int);
void subtract(int, int);
void multiply(int, int);
void divide(int, int);
```

Live Coding Part 1: Function Prototypes and Menu

```
#include <iostream>
using namespace std;

// Function prototypes
void showMenu();
void add(int, int);
void subtract(int, int);
void multiply(int, int);
void divide(int, int);

// Function definition
void showMenu()
{
    cout << "\n===== CALCULATOR MENU =====\n";
    cout << "1. Add\n";
    cout << "2. Subtract\n";
    cout << "3. Multiply\n";
    cout << "4. Divide\n";
    cout << "5. Exit\n";
}
```

- showMenu() does not require parameters.
- All five functions return void.
- Prototypes declare the functions before they are called.
- The menu function can be called repeatedly.

Why don't we write all the menu statements directly inside main()

- Modularity: Each function performs a specific task
- Readability: Keeps main() simple and organized.
- Reusability: Allows repeated calls to showMenu().
- Maintainability: Makes the menu easier to update

Part 2: Main Function and Menu Selection

```
int main()
{
    int choice, a, b;
    do
    {
        showMenu();

        cout << "Enter choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 4)
        {
            cout << "Enter two numbers: ";
            cin >> a >> b;
        }

        switch (choice)
        {
            case 1: add(a, b); break;
            case 2: subtract(a, b); break;
            case 3: multiply(a, b); break;
            case 4: divide(a, b); break;
            case 5:
                cout << "Exiting program...\n";
                break;
            default:
                cout << "Invalid choice, try again\n";
        }

    } while (choice != 5);
    return 0;
}
```

- Why use do-while instead of while?
- Why request two numbers only for choices 1–4?
- What does break do inside switch?
- What happens when the user enters 7?
- What happens when the user enters 5?

Part 3: Arithmetic Function Definitions

```
void add(int a, int b)
{
    cout << "Result: " << a + b << endl;
}

void subtract(int a, int b)
{
    cout << "Result: " << a - b << endl;
}

void multiply(int a, int b)
{
    cout << "Result: " << a * b << endl;
}

void divide(int a, int b)
{
    if (b == 0)
    {
        cout << "Cannot divide by zero\n";
        return;
    }

    cout << "Result: "
         << static_cast<double>(a) / b
         << endl;
}
```

- Each function performs one operation.
- a and b are passed by value.
- return; immediately exits the void function.
- Division by zero is checked before performing divisio
- `static_cast<double>(a)` allows decimal results.

Why can we use return; inside a void function but not return a / b

Complete Menu-Driven Calculator Program

```
#include <iostream>
using namespace std;

// Function prototypes
void showMenu();
void add(int, int);
void subtract(int, int);
void multiply(int, int);
void divide(int, int);

int main()
{
    int choice, a, b;

    do
    {
        showMenu();

        cout << "Enter choice: ";
        cin >> choice;

        if (choice >= 1 && choice <= 4)
        {
            cout << "Enter two numbers: ";
            cin >> a >> b;
        }

        switch (choice)
        {
            case 1:
                add(a, b);
                break;

            case 2:
                subtract(a, b);
                break;

            case 3:
                multiply(a, b);
                break;

            case 4:
                divide(a, b);
                break;

            case 5:
                cout << "Exiting program...\n";
                break;

            default:
                cout << "Invalid choice, try again\n";
        }

    } while (choice != 5);

    return 0;
}

// Display menu
void showMenu()
{
    cout << "\n===== CALCULATOR MENU =====\n";
    cout << "1. Add\n";
    cout << "2. Subtract\n";
    cout << "3. Multiply\n";
    cout << "4. Divide\n";
    cout << "5. Exit\n";
}

// Addition
void add(int a, int b)
{
    cout << "Result: " << a + b << endl;
}

// Subtraction
void subtract(int a, int b)
{
    cout << "Result: " << a - b << endl;
}

// Multiplication
void multiply(int a, int b)
{
    cout << "Result: " << a * b << endl;
}

// Division
void divide(int a, int b)
{
    if (b == 0)
    {
        cout << "Cannot divide by zero\n";
        return;
    }

    cout << "Result: "
         << static_cast<double>(a) / b
         << endl;
}
```

```
===== CALCULATOR MENU =====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit

Enter choice: 1
Enter two numbers: 10 20
Result: 30

===== CALCULATOR MENU =====
1. Add
2. Subtract
3. Multiply
4. Divide
5. Exit

Enter choice: 4
Enter two numbers: 10 0
Cannot divide by zero
```

In-class Exercise: Menu-Driven Grade Calculator

ite a C++ program that displays a menu allowing users to calculate the average, highest, or lowest of three test scc

Required menu:

```
===== GRADE CALCULATOR =====
1. Calculate Average
2. Find Highest Score
3. Find Lowest Score
4. Exit
```

- Required functions:

```
void showMenu();
double calculateAverage(int, int, int);
int findHighest(int, int, int);
int findLowest(int, int, int);
```

Requirements:

1. Display the menu repeatedly until the user selects Exit.
2. Ask for three test scores when options 1–3 are selected.
3. Call the corresponding function to perform the calculation.
4. Return the result from each calculation function and display it in main().
5. Handle invalid menu choices.
6. Use function prototypes, switch, and a loop.

Example execution:

```
===== GRADE CALCULATOR =====
1. Calculate Average
2. Find Highest Score
3. Find Lowest Score
4. Exit

Enter choice: 1
Enter three test scores: 80 90 100
Average Score: 90

===== GRADE CALCULATOR =====
1. Calculate Average
2. Find Highest Score
3. Find Lowest Score
4. Exit
Enter choice: 4
Exiting program...
```

Summary

Concepts reviewed

- Function prototypes and definitions
- Function calls, arguments, and parameters
- void versus value-returning functions
- Menu-driven programming using switch
- Repeating operations using do-while
- Handling invalid choices